Assignment 2 Business Analytics

Cell 1 : Importing raw data

Pandas for tables.
Matplotlib and Seaborn for charts.
Scikit learn for scaling
Decision tree and Graphviz for drawing the tree.

```
[7]
✓ 0s
    import pandas as pd, numpy as np
    import matplotlib.pyplot as plt, seaborn as sns
    from sklearn.preprocessing import StandardScaler
    from sklearn.tree import DecisionTreeClassifier, export_graphviz
    import graphviz
```

```
[8]
✓ 0s
raw_data = {
    'Student_ID': [1001, 1002, 1003, 1004, 1005, 1001, 1006, 1007, 1008, 1009],
    'County': ['Nakuru', 'Narok', 'Nakuru', 'Narok', 'Kakamega',
               'Nakuru', 'Nairobi', None, 'Narok', 'Nakuru'],
    'Study_Hours_Per_Week': [15, 8, 20, None, 12, 15, 30, 10, 25, 18],
    'Attendance_Percent': [85, 60, 92, 55, 78, 85, 95, 65, 88, None],
    'Fee_Balance_Ksh': [0, 15000, 0, 25000, 5000, 0, 0, 18000, 2000, 12000],
    'Previous_Grade': [72, 48, 85, 45, 65, 72, 90, 50, 82, 58],
    'Student_Type': ['Regular', 'Regular', 'Scholarship', 'Regular',
                     'Scholarship', 'Regular', 'Regular', 'Scholarship',
                     'Regular', 'Regular'],
    'Academic_Risk': [0, 1, 0, 1, 0, 0, 0, 1, 0, 1]
}
```

Cell 2 - Loading and Inspecting the Data

Here we turn the dictionary into a data frame table using pd.DataFrame(raw-data), df.shape tells the size to 10 rows and 8 columns, Df.types tells what type of data each column holds either numbers or texts and df.describe() gives quick statistics like average, smallest and largest value for each column

```
[9]
✓ 0s
```

```
df = pd.DataFrame(raw_data)
print(df.shape)     # (10, 8)
print(df.dtypes)
print(df.describe())
```

```
(10, 8)
Student_ID              int64
County                 object
Study_Hours_Per_Week  float64
Attendance_Percent    float64
Fee_Balance_Ksh         int64
Previous_Grade          int64
Student_Type           object
Academic_Risk           int64
dtype: object
        Student_ID  Study_Hours_Per_Week  Attendance_Percent  Fee_Balance_Ksh  \
count    10.000000             9.000000            9.000000        10.000000
mean   1004.600000            17.000000           78.111111      7700.000000
std       2.875181             7.123903           14.598326      9153.627089
min    1001.000000             8.000000           55.000000         0.000000
25%    1002.250000            12.000000           65.000000         0.000000
50%    1004.500000            15.000000           85.000000      3500.000000
75%    1006.750000            20.000000           88.000000     14250.000000
max    1009.000000            30.000000           95.000000     25000.000000

       Previous_Grade  Academic_Risk
count       10.000000      10.000000
mean        66.700000       0.400000
std         16.145519       0.516398
min         45.000000       0.000000
25%         52.000000       0.000000
50%         68.500000       0.000000
75%         79.500000       1.000000
max         90.000000       1.000000
```

Cell 3 : Data cleaning

isnull().sum() - counts the empty cells in each column and duplicated().sum() counts any repeated rows.
drop_duplicates() - delete the extra copies of data.

```
[10]
✓ 0s
print(df.isnull().sum())      # County 1, Study_Hours 1, Attendance 1
print(df.duplicated().sum())  # 1 duplicate row (Student_ID 1001)
df = df.drop_duplicates()     # 10 rows becomes 9

Student_ID              0
County                  1
Study_Hours_Per_Week    1
Attendance_Percent      1
Fee_Balance_Ksh         0
Previous_Grade          0
Student_Type            0
Academic_Risk           0
dtype: int64
1
```

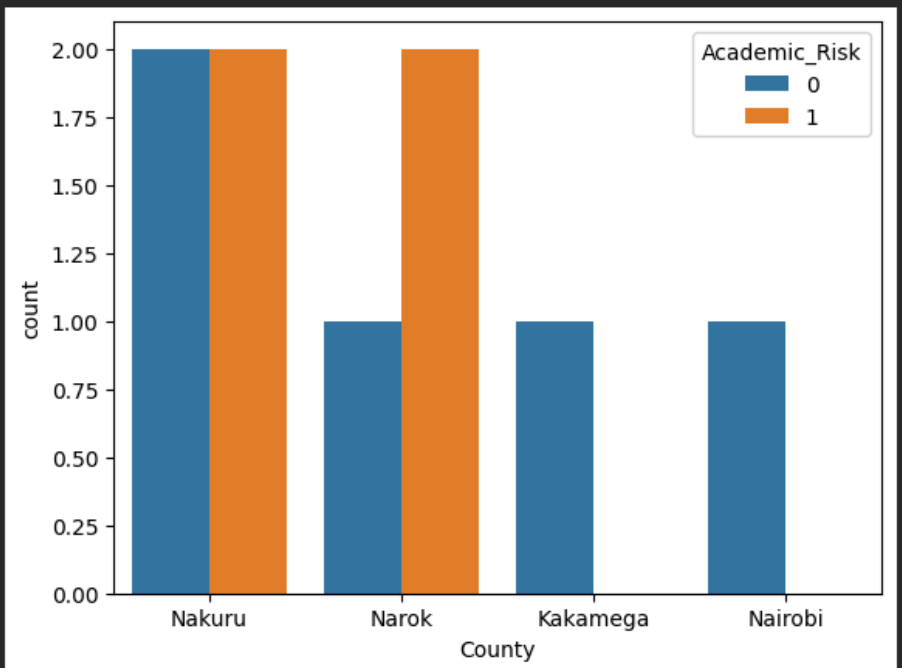
Cell 4 : EDA

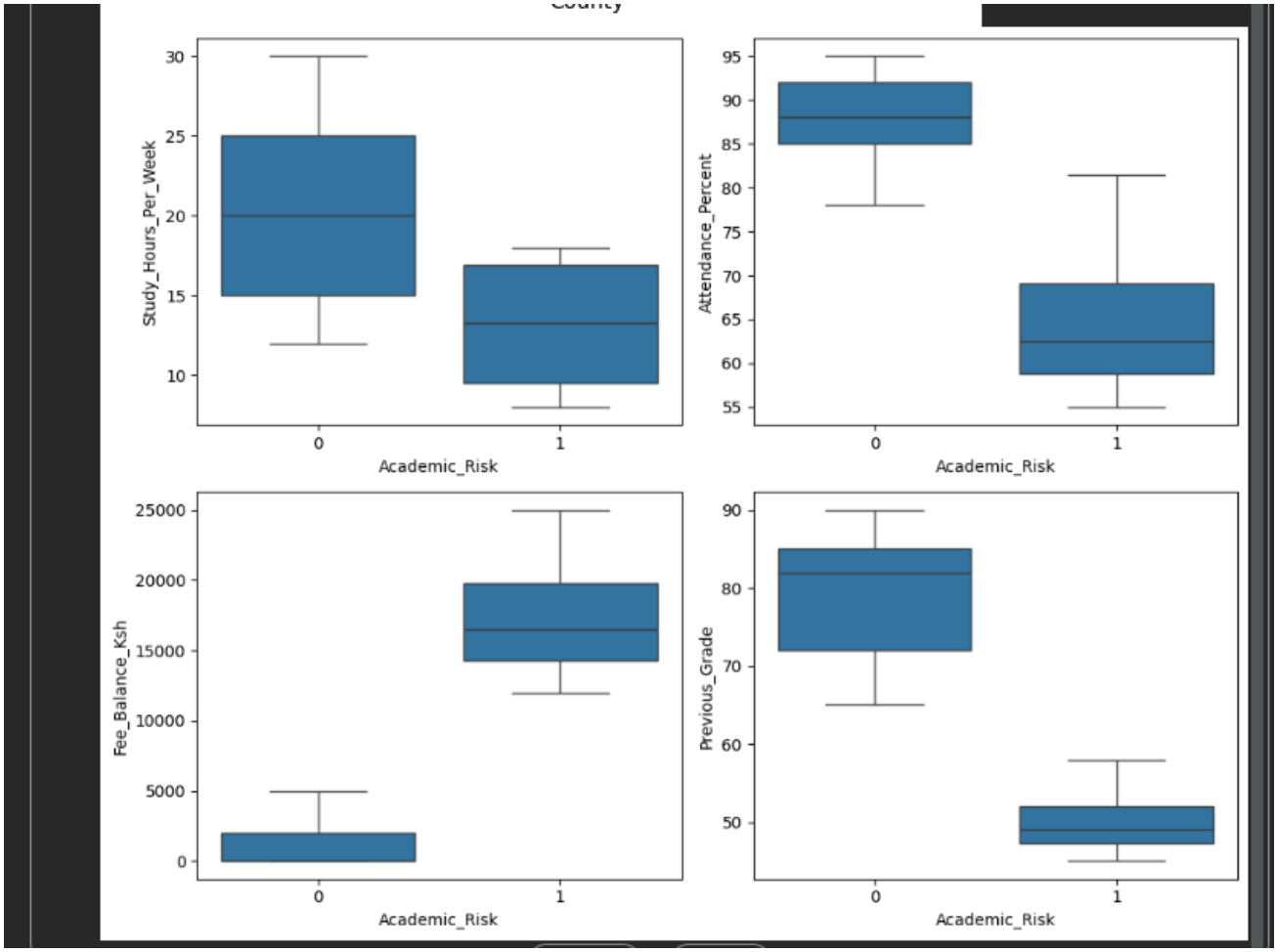
This cell draws charts to look for patterns. The count plot shows how many students in each county are at risk. The boxplots compare study hours, attendance, fee balance and previous grade for at risk and not at risk students. You are looking for differences between the two groups, such as at risk students having lower grades and higher fee balances.

```
[11]
✓ 0s
df['County'] = df['County'].fillna(df['County'].mode()[0])  # Nakuru
df['Study_Hours_Per_Week'] = df['Study_Hours_Per_Week'].fillna(df['Study_Hours_Per_Week'].median()
df['Attendance_Percent'] = df['Attendance_Percent'].fillna(df['Attendance_Percent'].median())
```

[12]
✓ 0s

```
sns.countplot(x='County', hue='Academic_Risk', data=df)
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
for ax, col in zip(axes.flat, ['Study_Hours_Per_Week','Attendance_Percent','Fee_Balance_Ksh','Prev
    sns.boxplot(x='Academic_Risk', y=col, data=df, ax=ax)
plt.tight_layout(); plt.show()
```





Cell 5: Feature Engineering

`Attendance_category-` sorts attendance into Low, Medium or High, which is easier to read than a raw percentage.

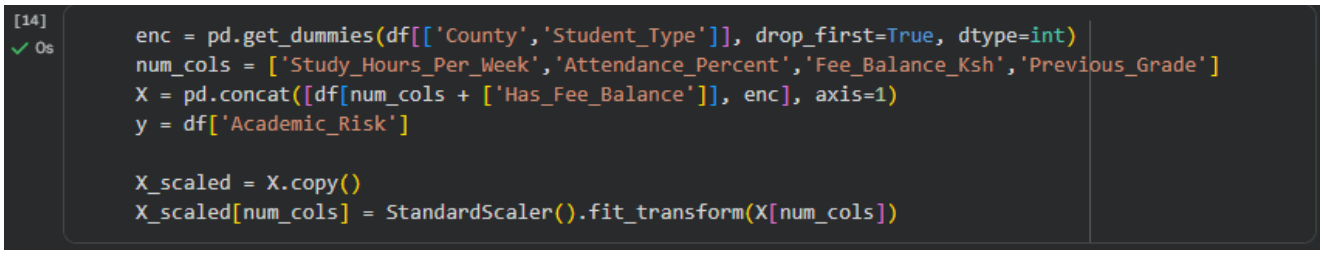
`Has_Fee_Balance` - 1 if the student owes money and 0 if not.

```
[13]
✓ 0s
    df['Attendance_Category'] = pd.cut(df['Attendance_Percent'], bins=[0,70,85,100],
                                       labels=['Low','Medium','High'])
    df['Has_Fee_Balance'] = (df['Fee_Balance_Ksh'] > 0).astype(int)
```

Cell 6: Encoding and scaling

Encoding: computers can't use words like "Nakuru" or "Scholarship", so `get_dummies` turns each category into a column of 0s and 1s.

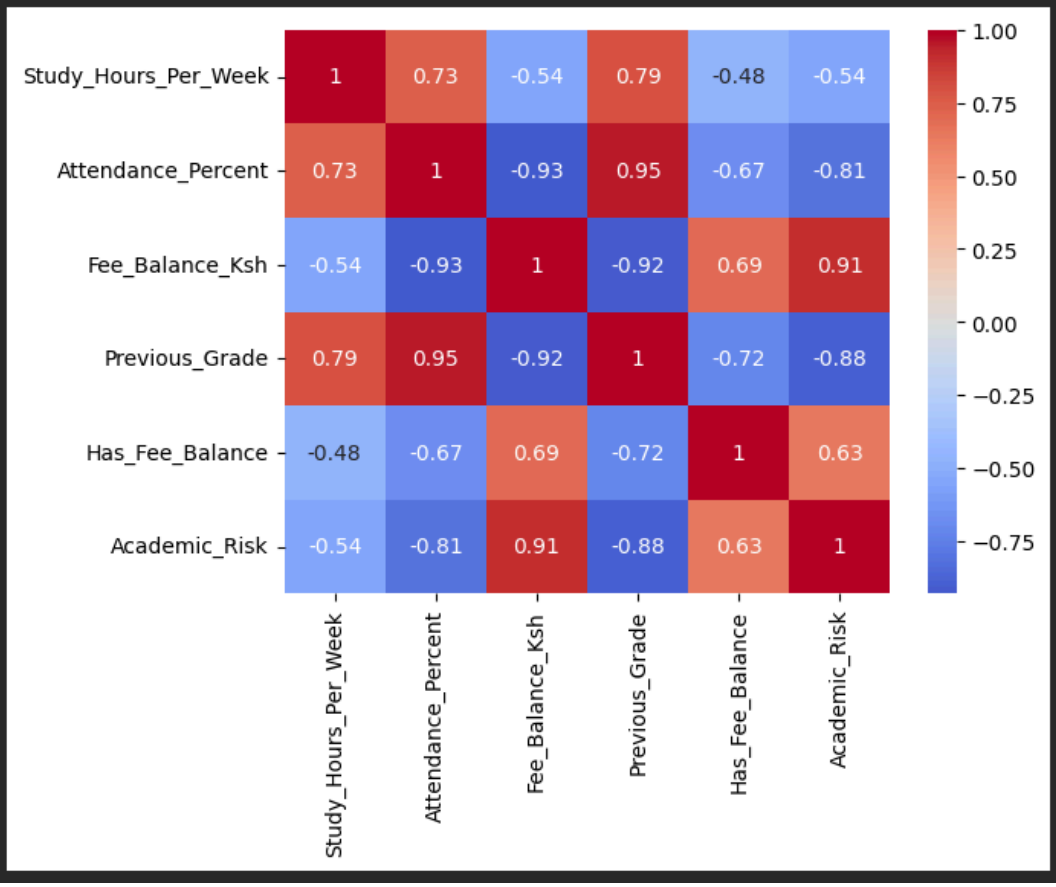
Scaling: `StandardScaler` puts the numeric columns on a similar scale. Fee balance is in thousands while study hours are in tens, and scaling stops the big numbers from overpowering the small ones.



Cell 7: Correlation heatmap

This draws a colored grid showing how strongly each pair of numbers is related. Values close to +1 mean "go up together" and values close to −1 mean "one goes up, the other goes down". Look at the Academic_Risk row to see which factor matters most. Fee balance is strongly positive, and previous grade and attendance are strongly negative.

```
[15]
✓ 0s
corr_cols = num_cols + ['Has_Fee_Balance', 'Academic_Risk']
sns.heatmap(df[corr_cols].corr(), annot=True, cmap='coolwarm', center=0)
plt.show()
```



Cell 8: Decision tree and Graphviz

- `DecisionTreeClassifier(max_depth=3)` builds a tree of yes/no questions, with at most 3 levels, to decide if a student is at risk.
- `.fit(X_scaled, y)` is the learning step, where the tree studies the data to find the best questions.
- `export_graphviz` and `graphviz.Source` turns the tree into a picture

```
dt = DecisionTreeClassifier(max_depth=3, random_state=42)
dt.fit(X_scaled, y)

dot = export_graphviz(dt, feature_names=X_scaled.columns,
                      class_names=['Not at risk','At risk'],
                      filled=True, rounded=True)
graphviz.Source(dot).render('tree', format='png', view=True)
```

```
'tree.png'
```